Synthetic vs. real RGB-D data in pose estimation and segmentation tasks for aluminium profiles and brackets

Francesco Baldassarre

Abstract

Robots that handle aluminium profiles and brackets are often developed on synthetic depth images, which are usually rendered as perfect geometry. This study asks whether a perception method should be tuned on such ideal depth, or on realistic depth that contains the errors of a real camera. I build a pipeline that generates labelled RGB-D scenes of parts dropped onto a table, and a model of the RealSense D435i depth camera whose parameters come from the manufacturer and from published measurements. Two tasks, pose estimation and the separation of the parts from the table, are tuned and tested on ideal depth (A), tuned on ideal and tested on realistic depth (B), or tuned and tested on realistic depth (C). For pose estimation, realistic depth leaves the success rate almost unchanged (99.3 % and 99.0 % of 420 parts), but the median error of the pose centre grows from 0.01 to 1.9 mm. Almost all of this loss comes from the higher noise on aluminium, and tuning on realistic depth does not recover it. In addition, if we consider that aluminium seen at more than 45° returned no depth due to reflections, the success rate would fall to 77 %. For the segmentation task, a method tuned on ideal images still finds almost every part on realistic depth scenarios, but its pixel precision drops from 100 % to 28 %. In both tasks, ideal depth predicts whether the parts are found, but not how precisely.

1 Introduction

A robot that picks and assembles aluminium profiles and brackets will mostly perceive them through a depth camera. Aluminium is a difficult material for such cameras. It is shiny, its features (slots, thin walls, sharp edges) are only a few pixels wide at working distance, and the errors of the camera depend on the geometry and the viewpoint. Synthetic data are attractive for such parts, because their geometry is known exactly and labels come for free, but in this way a method may be tuned on data that no real sensor will ever see due to noise and errors.

Research questions. *Does modelling the imperfections of the depth camera change the outcome of a perception step developed on synthetic data? Which imperfections matter the most?*
This study answers them with a comparison among three conditions, that differ only in the depth images used (Figure 1). In **A**, the method is tuned and tested on ideal depth. In **B** it is tuned on ideal depth and tested on realistic depth. In **C** it is tuned and tested on realistic depth. If B differs from A, the camera model matters. If C is better than B, it pays off to model the camera during development. Each modelled effects is switched on independently from the others to show its own difference on the final result.

The experiment is organized into two steps. **Step 1** uses the camera model of Section 3, in which every parameter has a source for this camera. **Step 2** consider two additional effects that have no published value for this camera, and thus tries them out at three different strengths each. These latter are holes on aluminium seen at an angle, and noise that is shared by neighbouring pixels. The 6D pose estimation of each part is tested on both Steps 1 and 2, whereas the part/table segmentation only consider Step 1 data. The code runs on a laptop inside a Docker container and is available at `https://github.com/baldassarreFr/RGB-D_vision`, with the raw results of every run and the detailed documentation of the implementation.

2 Synthetic scene pipeline

Each scene starts from a random seed, which chooses the parts and where they are dropped. A physics simulator (PyBullet [9]) drops them onto a table, so that they rest in realistic and varied poses, and then Open3D ray casting [10] computes the ideal depth, the instance label of every pixel (table, profile or bracket) and the surface normals. These are stored with the true poses and a shaded colour image. The full pipeline is reported in Figure 1, while Figure 2 shows a sampled scene. The camera model of Section 3 is applied afterwards, so every camera variant sees exactly the same scenes.

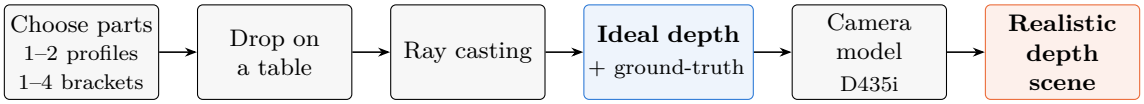


Figure 1. The synthetic scene pipeline. The camera model adds noise, stereo shadows and shifted edges (Step 1), and in Step 2 also holes or correlated noise.

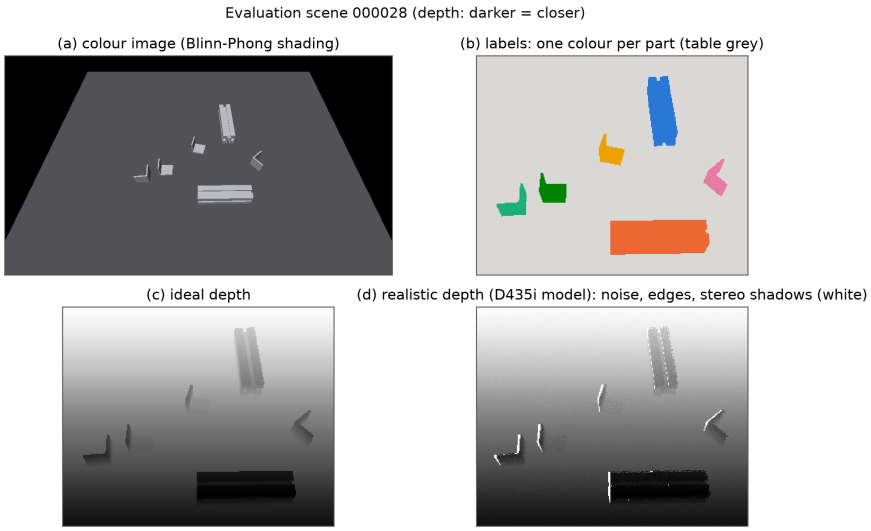


Figure 2. One evaluation scene. (a) Shaded colour image, (b) instance labels, (c) ideal depth, (d) realistic depth from the camera model of Section 3 (white = no depth).

Camera. The reference camera is the Intel RealSense D435i. It is widely used in robotic applications. For example, is the head camera of the Unitree G1 humanoid [6]. It is an active stereo camera, featuring two infrared sensors 50 mm apart see the scene, and a projector between them adds a pattern of dots [2, 5]. The D435 has the same depth specifications [3], so its published measurements apply. Depth is simulated at 848 × 480 pixels, the best depth resolution according to Intel [4], with an ideal pinhole model and a horizontal field of view of 87°. The camera is located so that it looks at the centre of the table from 0.65 m, at 57.5° below the horizontal, roughly the view of a humanoid's head onto a workbench. One pixel then covers about 1.5 mm.

Parts. Both parts are built by extruding a 2D cross-section [7, 8]. The T-slot profile is 40×40 mm and 150 mm long, with one slot per side and a central bore. The angle bracket is an L with 40 mm legs, 4 mm walls and a width of 36 mm.

3 Camera model

3.1 Why aluminium is hard for a stereo camera

A stereo camera like the D435i finds depth by matching small image patches between its two infrared images. A point at depth z appears shifted between them by the disparity $f_x\, b/z$, where f_x is the

focal length in pixels and b the baseline. Matching finds this shift up to a fraction of a pixel, the subpixel accuracy σ_s, which Intel's calculations take as 0.08 px [4]. So the depth error grows with the square of the distance,

$$\sigma_z = \frac{z^2\,\sigma_s}{f_x\,b}. \tag{1}$$

This gives 1.5 mm at 0.65 m, in line with an independent test of the same camera [12]. The aluminium parts are usually some mm thin, meaning that in the image their walls occupy only a few pixels.

Another problem related to metallic objects is that matching assumes that a point looks the same from both sensors, and shiny aluminium breaks this. In some particular lighting conditions, it reflects the projected dots away from the camera like a mirror, resulting in points with no depth.

3.2 Effects in the model

Table 1 lists every effect considered, and the step in which it is applied. The difference among the two steps is that Step 1 contains only the effects with a published value for this camera, while Step 2 explores two additional effects for which I found no public value, but didn't want to leave out. Therefore, I test each of them at three strengths, to measure their impact at different levels. Figure 3 shows the Step 1 effects on one sampled scene.

Effect	Why it happens	What the model does	Value and source
Step 1			
Shifted edges	At an object edge, the matching patch covers two depths, so edges are never exact.	Each pixel on an object edge takes the depth of its neighbour with probability 50 %. Edges grow and shrink equally often.	Up to 1 px for the D435 [13]. The 50 % is my choice, as the source does not specify.
Stereo shadows	A point must be seen by both sensors to get a depth. If one of them cannot see it, it gets no depth.	A pixel keeps its depth only if the right sensor also sees the same point. Depth is lost on the left side of objects, in a band on the left of the image and inside slots.	50 mm baseline [2].
Depth noise	Matching finds the shift between the two images only up to a fraction of a pixel.	Each pixel moves along its viewing ray by a random Gaussian error of size σ_z (Eq. 1)	$\sigma_s = 0.08$ px [4, 12].
	Shiny aluminium is matched less precisely than matte surfaces.	The noise is 2.9 times larger on aluminium parts than on matte surfaces (as the table).	Measured for the D435 on aluminium and matte plates [13, Fig. 6].
Step 2			
Holes on aluminium	A shiny surface tilted away from the camera reflects the projected dots elsewhere, so the matcher finds no texture.	Every aluminium pixel tilted by more than an angle θ_c from the line of sight loses its depth. The table is never affected.	$\theta_c = 75°$, 45° and 30°.
Correlated noise	The matcher combines neighbouring pixels, so real errors come in patches [5].	Same noise size as in Step 1, but neighbouring pixels move together.	Patch size of 2, 5 or 10 px.

Table 1. Effects considered for the camera model. The Step 1 effects are applied in the order listed and together give the realistic depth. Each Step 2 effect is added to the realistic depth on its own.

Effects left out. Four effects are not in the model. Noise that grows with the viewing angle is expected, because surfaces seen at a grazing angle return less light. But published model for this camera [14] gives errors larger than the parts on our scenes.

Shiny surfaces also show highlights, the bright reflections of a light. The two sensors look from different points, so each sees a highlight at a slightly different place on the surface. The matcher can pair the two highlights as if they were the same point, which gives a wrong depth without any warning. How often this happens was never measured for this camera. Simulating it would require rendering the two infrared images with reflections and running a stereo matcher on them.

Finally, projector shadows and calibration errors are not considered in this experiment.

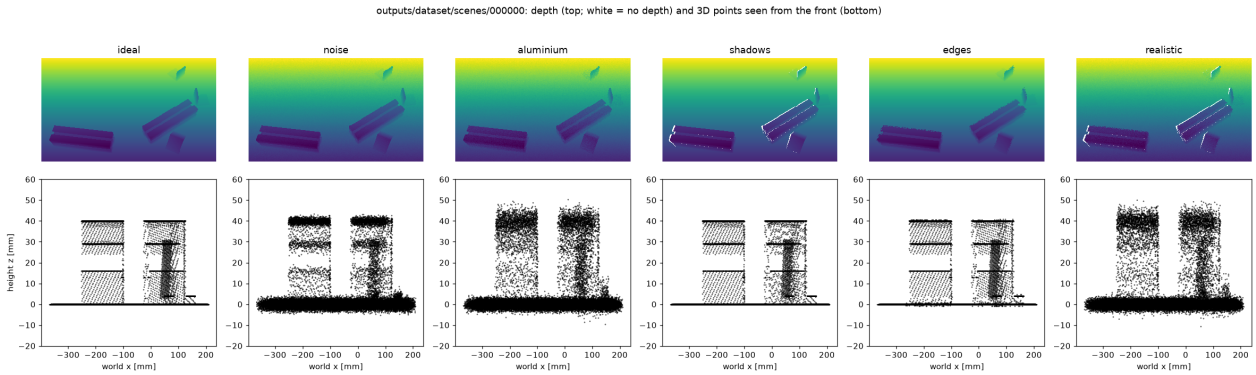


Figure 3. One scene in the six camera variants of Step 1. Top, depth (white = no depth). Bottom, the 3D points seen from the front.

4 Experimental design

Data. All scenes come from the pipeline of Section 2. Each scene is rendered once as ideal depth, and the camera model then turns it into realistic depth and into one variant for each effect alone. Parameters are tuned on 20 generated scenes with 93 parts in total, and then the evaluation is performed on 100 different scenes with 420 total parts (267 brackets, 153 profiles). All camera variants use the same scenes, so differences between conditions come from the camera alone.

Task 1: pose estimation. For grasping a part, a robot needs its position and orientation. In this project, to estimate it the points of each part are cut from the depth image with the true labels, so that only the pose step is tested. Then, they are aligned to the part's CAD model using FPFH feature descriptor to match pairs of points, and then RANSAC to give a first coarse pose [10, 15, 16]. Finally, ICP refines it [17, 18]. With the settings of the Open3D tutorial, the method failed on our parts. Small tests on the tuning scenes led to four changes, used in all three conditions:

- RANSAC accepts a match within 1.0 voxel instead of 1.5, which was wider than the 4 mm wall of a bracket.
- A wide ICP pass runs before the tight one, which alone could not correct the first pose.
- Five attempts are made per part, and a depth check keeps the one that best explains the measured depth [20, 21].
- Two scales are tuned, the voxel size v for the shape detail and the tolerance d for the noise.

Metrics. ADD-S is the mean distance between the model in the estimated pose and in the true pose, ignoring the symmetries of the parts [19, 22]. A pose is correct if ADD-S is below 10 % of the part's size. This threshold is generous, since a profile slid 50 mm along its axis still counts as correct. So the report also gives the error of the part centre and the ADD-S AUC, the mean of $\max(0,\ 1 - \text{ADD-S}/\text{threshold})$, which rewards accuracy [22].

Tuning. The method has two settings, the voxel size v and the tolerance d. For each part type, I try every combination of a few values of v and d on all parts of the 20 tuning scenes, and keep the one with the highest ADD-S AUC. This search is done twice: on ideal depth it gives the settings of A and B, and on realistic depth those of C.

Evaluation. After tuning, the settings are fixed and the method runs once on the 100 unseen scenes. A uses the settings tuned on ideal depth and tests them on ideal images. B uses the same settings on realistic images. C uses the settings tuned on realistic depth, also on realistic images. As an ablation study, the settings of A are also tested on each camera effect alone.

Task 2: segmentation. Before its pose can be estimated, a part must be separated from the rest of the scene. Here, I test the simplest method that uses depth only to decide whether a pixel represent the table or a part, without labelling each part separately. To do so, a plane is fitted to the table with RANSAC, and every pixel more than a threshold t above it is marked as part. Touching parts are not separated from each other. A part counts as found if at least half of its pixels are marked. The pixel precision is the share of marked pixels that really belong to a part. The threshold is tuned by the F1 score of these two numbers. On ideal images, $t = 1$, 2 and 3 mm are all perfect. B uses 2 mm and is also reported with the other two.

5 Results

5.1 Pose estimation

This section reports the results for the pose estimation and a brief comparison among the setups.

5.1.1 Step 1

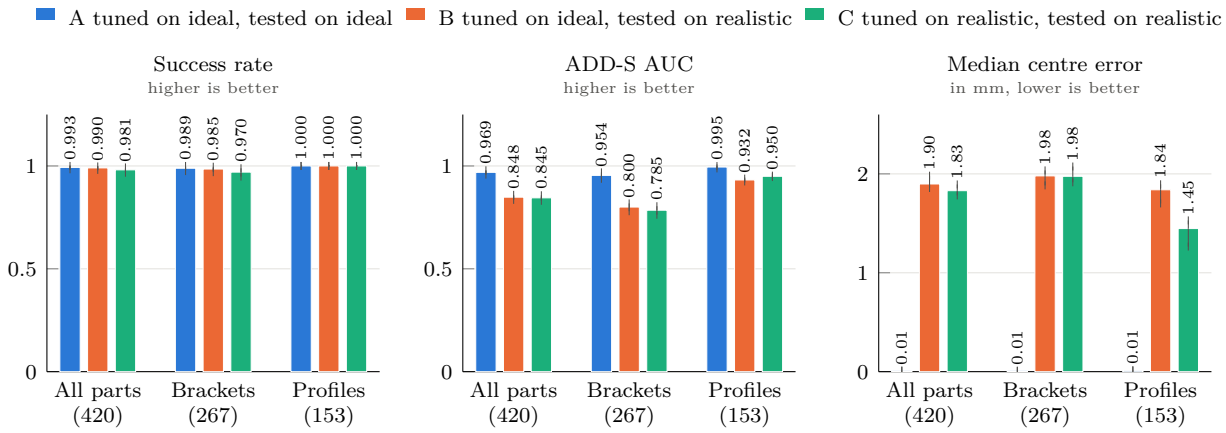


Figure 4. Pose estimation on the 100 test scenes.

A vs. B. With the same tuning on ideal data, realistic depth does not change the success rate in a significative way (Figure 4). Overall, only 3 of the 420 parts succeed on ideal depth and fail on realistic depth, and 2 parts do the opposite, so the method is still able to find the parts. What really changes is the precision of how exactly it places them. On ideal depth, almost every part is placed exactly. Indeed, the median error of the part centre is 0.01 mm, and 87 % of the parts are within 1 mm. On the other hand, on realistic depth the median error grows to 1.9 mm, and only 5 % of the parts are within 1 mm (Figure 5). Almost every part is placed a little off, and 85 % of them are between 1 and 3 mm away from the true centre. The orientation is also off, with a median error of about 2° instead of almost zero in setup A. The error in millimetres is similar for brackets and profiles, but since a bracket is smaller, the AUC is lower for it.

Figure 6 shows an example of the full pose estimation pipeline on a bracket on both ideal and realistic depth. At the end the estimated pose lies about one pixel away from the true one.

B vs. C. Tuning on realistic depth does not change the success rate. The centre error, however, goes down. Over all parts, the median drops from 1.90 to 1.83 mm, and almost all of this gain comes from the profiles. In C, 124 of the 153 profiles are placed better than in B, and their median error drops from 1.84 to 1.45 mm. In B, 10 profiles are misplaced, between 5 and 16 mm from the true

centre. They still count as correct, because their ADD-S stays below the success threshold. In C, only one profile is still that far off. On the other hand, the brackets do not gain, and their median error stays unchanged.

Which effect matters the most? Figure 7 adds one camera effect at a time to ideal depth, with the settings tuned on ideal depth. Stereo shadows remove depth next to the parts, but the result does not change. Shifted edges and the noise of a matte surface each move the centre by about half a millimetre. Both hurt the brackets more than the profiles. The shifted edges, for example, add 0.8 mm to the brackets and almost nothing to the profiles. The higher noise on aluminium, alone, moves the centre by about 2 mm. This is about as much as all the effects together. So almost the whole loss comes from one number of the camera model: the noise factor of 2.9 for aluminium.

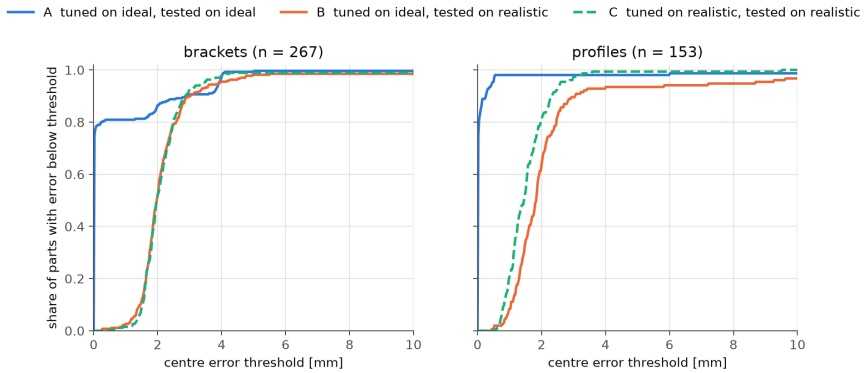


Figure 5. Share of parts whose centre error is below a threshold.

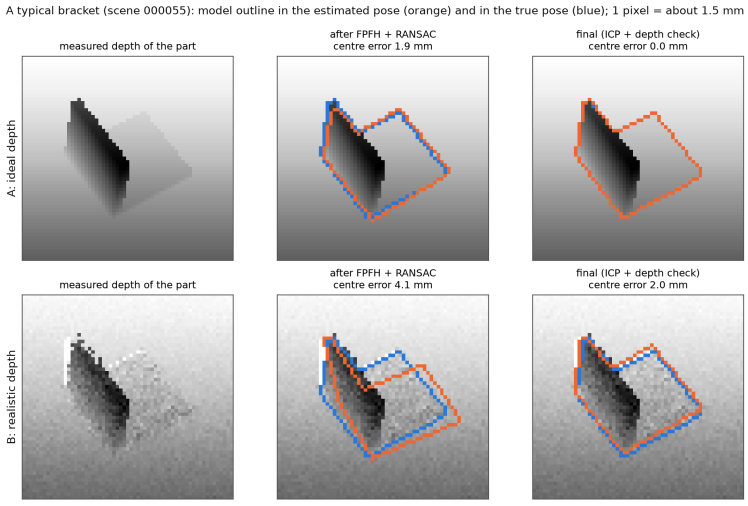


Figure 6. Example of pose estimation pipeline on a bracket on both ideal depth (top), and realistic depth (bottom), using a method tuned on ideal scenes.

5.1.2 Step 2

Step 2 pushes the camera model further by adding two effects that are expected on shiny aluminium, but have no published value for this camera, each at three strengths (Table 1). Each effect is added on top of the realistic depth of Step 1. B keeps the settings tuned on ideal depth, and C is tuned again on each variant.

Holes remove the depth of aluminium surfaces seen at an angle steeper than θ_c. Unlike the effects of Step 1, they make the method fail (Figure 8). With $\theta_c = 75°$, 20 brackets are lost and 1 is gained compared with Step 1. With 45°, the success rate falls to 77 %, and 1/3 of bracket is lost. Almost all profiles are still found, but their median centre error jumps from 1.8 to 10.5 mm. With

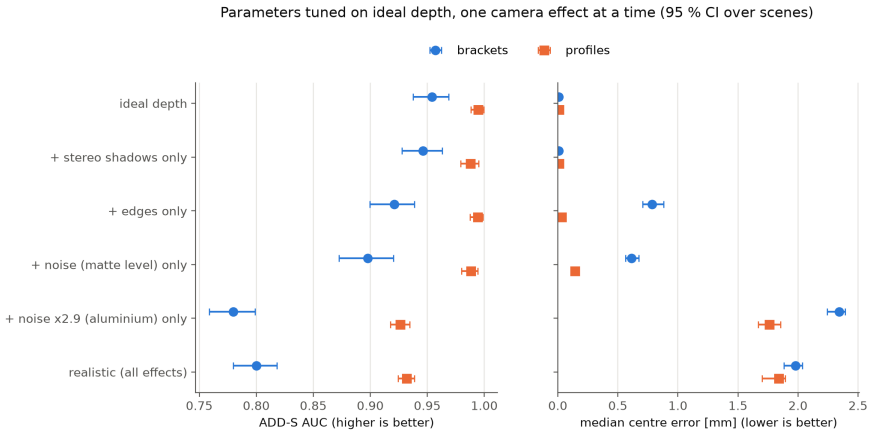


Figure 7. Parameters tuned on ideal depth, tested with one camera effect at a time.

30° the success rate falls to 25 %, because 258 of the 420 parts keep fewer than 10 points and get no estimate at all. Unlike in Step 1, tuning on realistic scenes helps a lot here. With 75° the success rate is back at 99 %. With 45° it halves the centre error of the profiles.

Correlated noise makes nearby pixels err together, over patches of 2, 5 or 10 pixels. It does not change the success rate, but it makes the estimate less accurate. The median centre error grows from 1.9 mm in Step 1 to 2.1, 2.3 and 2.5 mm as the patches grow. As in Step 1, the brackets suffer more. With 10-pixel patches their error grows by 0.7 mm, and that of the profiles by 0.3 mm. In this case, tuning on realistic scenes changes little.

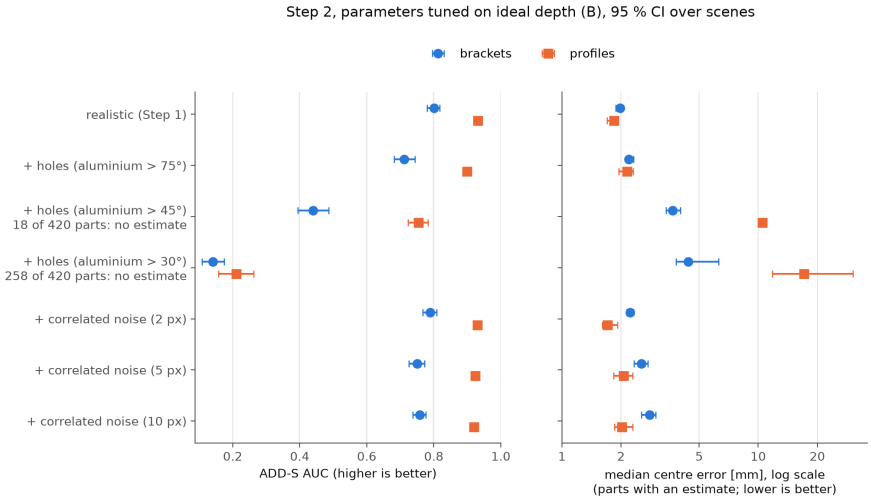


Figure 8. Step 2, condition B. Top row, realistic depth of Step 1. Centre error on a log scale, for parts with an estimate. The figure for C is in the repository.

5.2 Segmentation

The table plane is found almost exactly in every condition. On ideal depth (A) every part is found and every marked pixel belongs to a part. On realistic depth with the same threshold (B), 419 of the 420 parts are still found, but the method also marks a large part of the table as part. Of all the pixels it marks as part, only 28 % really belong to a part, while the other 72 % are table. The three thresholds that are equally perfect on ideal depth give a precision between 10 and 65 % on realistic depth. The ablation shows that the noise causes this loss (29 % with noise alone), while shadows and edges hardly matter.

Unlike in pose estimation, tuning on realistic depth helps a lot here. The tuning picks a higher threshold, 4 instead of 2 mm, so the noise lifts far fewer table pixels above it. Over the 100 test

scenes, the precision rises from 28 to 90 %. This gain has a small cost: a higher threshold also cuts the lowest pixels of each part, so C misses 6 brackets that A finds.

Figure 9 shows one test scene. Note that the numbers above its panels refer to this scene only.

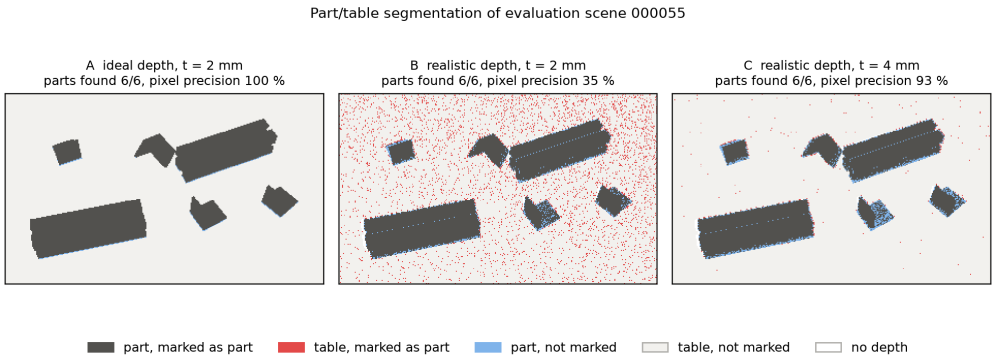


Figure 9. Segmentation of one test scene. With the threshold tuned on ideal depth, realistic depth (B) marks many table pixels as part. The threshold tuned on realistic depth (C) removes most of them, rising the per-pixel precision.

6 Conclusions and limitations

With the camera model of Step 1, modelling the camera does not change whether the pose estimator finds the parts. It changes how exactly it places them. On ideal depth almost every part is placed within 1 mm, on realistic depth almost every part is 1 to 3 mm off. So what ideal data promise depends on the accuracy a task needs. For a task that needs every part within 2 mm of its true position, a method tested on ideal depth seems good enough for 90 % of the parts. However, when tested on realistic depth, it is good enough for only 56 % (Figure 5). Almost all of this difference comes from the noise factor of 2.9 for aluminium.

Modelling the error is useful in two ways. First, it tells in advance how well a method will work on the real camera. Second, it can be used to tune the method. How much the second use helps depends on the task. For example, for the segmentation task it helps a lot, since on ideal depth three thresholds are equally perfect, so ideal data cannot even tell which one to use. Tuning the method on realistic depth rises the precision from 28 to 90 %.

Limitations and future directions. Realistic is not real. Without real images, the experiment shows how much the conclusions depend on the camera model, not that the model is right. Pose estimation uses the true labels to cut out the parts, and the two tasks were not chained. Combining parts segmentation and pose estimation would be an interesting future direction to simulate an even more realistic scenario.

7 Use of AI tools

I used Claude (Anthropic) to help understanding concepts, writing the code one module at a time, and rewriting the report with scientific rigor. The design decisions were mine, among them the parametric camera model, the part sizes, the three conditions and the code structure. I corrected the tool several times. An example of this is during the modelling of stereo shadow, where the AI check used a search window too small for an upright profile. Its edge detector would have missed the edge of a 4 mm bracket. Additionally, AI proposed camera values without a source, and I removed them.

References

[1] Intel, *Intel RealSense Depth Camera D435i – Product Specifications*. `https://www.intel.com/content/www/us/en/products/sku/190004/intel-realsense-depth-camera-d435i/specifications.html`

[2] RealSense, *D400 Series Product Family Datasheet*, Mar. 2026. https://www.realsenseai.com/wp-content/uploads/2026/03/RealSense-D400-Series-Datasheet-Mar-2026.pdf

[3] RealSense, *Depth accuracy for Intel RealSense D400 series cameras*. https://support.realsenseai.com/hc/en-us/articles/360059129453

[4] A. Grunnet-Jepsen, J. N. Sweetser, J. Woodfill, *Tuning depth cameras for best performance*, RealSense white paper, rev. 2.0. https://dev.realsenseai.com/?p=41439

[5] L. Keselman, J. Iselin Woodfill, A. Grunnet-Jepsen, A. Bhowmik, "Intel RealSense stereoscopic depth cameras," *CVPR Workshops*, 2017. arXiv:1705.05548.

[6] Unitree Robotics, *Unitree G1 humanoid robot*. https://www.unitree.com/g1

[7] S. Gillies et al., *Shapely*. https://github.com/shapely/shapely

[8] M. Dawson-Haggerty et al., *trimesh*. https://trimesh.org

[9] E. Coumans, Y. Bai, *PyBullet, a Python module for physics simulation for games, robotics and machine learning*, 2016–2021. https://pybullet.org

[10] Q.-Y. Zhou, J. Park, V. Koltun, "Open3D: A modern library for 3D data processing," arXiv:1801.09847, 2018.

[11] J. F. Blinn, "Models of light reflection for computer synthesized pictures," *SIGGRAPH*, 1977.

[12] L. Rustler, V. Volprecht, M. Hoffmann, "Empirical comparison of four stereoscopic depth sensing cameras for robotics applications," *IEEE Access*, vol. 13, pp. 67564–67577, 2025.

[13] G. Halmetschlager-Funek, M. Suchi, M. Kampel, M. Vincze, "An empirical evaluation of ten depth cameras," *IEEE Robotics & Automation Magazine*, vol. 26, no. 1, pp. 67–77, 2019.

[14] M. S. Ahn, H. Chae, D. Noh, H. Nam, D. Hong, "Analysis and noise modeling of the Intel RealSense D435 for mobile robots," *Int. Conf. on Ubiquitous Robots (UR)*, pp. 707–711, 2019.

[15] R. B. Rusu, N. Blodow, M. Beetz, "Fast point feature histograms (FPFH) for 3D registration," *IEEE ICRA*, pp. 3212–3217, 2009.

[16] M. A. Fischler, R. C. Bolles, "Random sample consensus," *Commun. ACM*, 24(6):381–395, 1981.

[17] P. J. Besl, N. D. McKay, "A method for registration of 3-D shapes," *IEEE TPAMI*, 14(2):239–256, 1992.

[18] Y. Chen, G. Medioni, "Object modelling by registration of multiple range images," *Image Vis. Comput.*, 10(3):145–155, 1992.

[19] S. Hinterstoisser et al., "Model based training, detection and pose estimation of texture-less 3D objects in heavily cluttered scenes," *ACCV*, LNCS 7724, pp. 548–562, 2012.

[20] A. Aldoma, F. Tombari, L. Di Stefano, M. Vincze, "A global hypotheses verification method for 3D object recognition," *ECCV*, 2012.

[21] T. Hodaň et al., "BOP: benchmark for 6D object pose estimation," *ECCV*, 2018.

[22] Y. Xiang, T. Schmidt, V. Narayanan, D. Fox, "PoseCNN: a convolutional neural network for 6D object pose estimation in cluttered scenes," *Robotics: Science and Systems*, 2018.